COMP 452 Assignment 2, Part 2 (Ant Colony) Description File

Bryce Dixon (3397649) Jan 2026

TABLE OF CONTENTS:

- PART 1: Logic and Game Design - Page 1
- PART 2: Game Compilation Instructions - Page 6
- PART 3: Game Play Instructions - Bottom Page 6
- PART 4: State Machine Description - Page 7
- PART 5: Current Game Bugs/ Problems - Page 9

PART 1: Logic and Game Design

DESIGN GOAL:

My goal with the program was for it to be easy to use, with nice colours which make it easy to visualize the AI logic used in the game. I wanted both ant states and state transitions to be easily seen and differentiated from one another, while also moving at a speed and in a manner that looks similar to ant movement. Animations (colour and size changes) were added to enhance this.

CLASS STRUCTURE OVERVIEW:

- Main
- GridCreator
    - GridCell
- AntObject
    - AntBehaviours
    - CollisionDetection
    - AStarMain2
        - AStarNode2
        - AStarListManager2
    - AntStateMachine
        - AntObject
            - WanderFood
            - WanderWater
            - FindPath
            - ReturnHome
            - Dead

GAME LOGIC OVERVIEW

GRID CREATION LOGIC

- The grid creation uses a Pane object and then manually places the GridCells by incrementing row and column values.
- Grid cells are a Rectangle object.
- The first GridCell (row = 0, col = 0) is always the home colony.
- All GridCells within 2 rows and/or 2 columns of the home colony are set to be Ground. This was done to:
    - A. Not allow a poison or food cell to immediately take over if it was right beside the colony.
    - B. Reduce the risk of the home colony being completely blocked off by poison cells. This is still possible though highly improbable.
- Outside of this, cells are randomly assigned Poison, Food, Water, or Ground types.
    - This is done using basic probability, where Ground has the highest probability, while Poison, Food, and Water have equal probability of being chosen.

USER ENTRY LOGIC

- The user enters the number of ants they would like to see used in the demonstration and click submit when ready.
- I have this limited to an integer between 1 and 1000. The program works for larger numbers, but I wanted to have an upper limit to avoid outrageous inputs and 1000 is already very busy for this grid size.
- This also will not run if the user enters anything outside of this range OR anything that is not an integer.

ANT LOGIC

- Ants are a Circle object.
- Ants are stored in an ArrayList<>, which is iterated through once per game loop.
- On each loop the program checks if any of the ants have their boolean for "requestingNewAnt" set. The ants set this when they return home to ask for a new ant to be born. They create a new ant and add it to the "antsToAdd" list.
- Then the update method for the ant is run. This runs the ant's state machine, which is described later in this document.
- After all ants are iterated through, the game checks if the "antsToAdd" list is empty, if it is not, the game adds all ants from this list to the main antsList.
    - This was done as if I tried to add them while the main antsList was being iterated, it threw a ConcurrentModification error, so I did it after the iteration is complete.

ANT COLLISION LOGIC

- Ant collision handling is handled within a CollisionDetection class which each ant implements.
- Ants do not detect collisions with one another. Initially I had the ants colliding with one another, however, this looked unnatural, as when watching videos of ants online they crawl over top of one another.

- Grid Border collisions simply check if the ant is within its radius of the border, if it is, its velocity is flipped in whatever direction points away from the wall (i.e. if it hits a border on the left or right, its x velocity is multiplied by -1). This looks similar to the twitchy movements of ants, as I did not feel a toroidal environment was appropriate for this simulator.
- Ant GridCell collisions use a circle-rectangle Axis Aligned Boundary Box (AABB) algorithm. This algorithm first finds all the cells that the ant could possibly be touching. For each of these cells, it finds the border of the rectangle which is closest to the ant, then it checks if this border is within the ants radius. If it is, we have a collision. This is used to detect if the ant has collided with Food, Water, or Poison cells during the two Wander states.
    - References and more detailed explanations for this are in my source code.

ANT WANDER LOGIC

- The choice of ant speed and rotation was chosen from trial and error, trying to get as close to ant-like movement as possible.
- A random rotation is chosen between 0 and 20 degrees, and the ants x and y velocities are updated to represent this. This avoids rapid turns but does appear as the "jittery" type of movement ants often have.

ANT PATH FINDING

- The ants in the game use a similar A* search algorithm to what I created and described in Assignment 2 Part 1, with a few changes:
    - Each ant stores their own copy of the A* algorithm.
    - Since ants can continually use their A* algorithm, I had to make sure that I cleared all open and closed lists and reset all variables as to not throw off successive path finding attempts.
    - HASHMAP:
        - Within this, they store a HashMap<GridCell, AStarNode> which uses the grid cell, which is the cell from the main game grid, as the key and an AStarNode as the value.
            - This was done as initially I had the GridCell storing an AStarNode within it, but with all ants using the same Grid, they were overwriting one another's H, G, and parent values in the AStarNode. So now each ant holds its own AStar node that references a particular GridCell, meaning no ant modifies the shared map data.
            - The HashMap also only stores GridCells that the ant uses, it only creates entries as they are requested, saving time and memory.
    - PATH SMOOTHING:
        - I added path smoothing to the path found by the A* algorithm, which uses a Bresenham Line Algorithm (references and more details are included in my source code).

- As is described in the course textbook, I loop through the found path, and check a line of sight between the current cell and the next cell. I iterate through the next cells until a line of sight does not exist (i.e. we hit a Poison cell). I then set the current cell to the last reachable cell and continue this.
- This alone 'worked', but it resulted in a lot of corner cutting where the ant would walk right through a poison cell on its smooth path home.
- CORNER CUTTING FIX:
    - I added a check where if the path followed by the line drawing algorithm went diagonal, it would check if a line of sight existed between the previous cell and the next cell on either side of the diagonal. This was done by first tracking the previous cell and next cell, then checking if these were both on different columns and rows (suggesting a corner occurred).
    - If they were, I would check the squares on either side of the corner for poison to account for right, left, up, and down corners (Images 1 & 2).

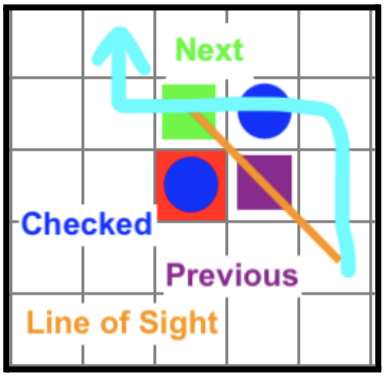

IMAGE 1: Diagonal checks

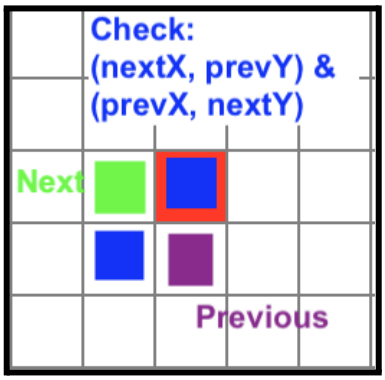

IMAGE 2: Diagonal checks 2

    - This improved slightly, but corner cutting was still an issue as the line was not accounting for the width of the ant character. This seemingly was occurring on shallower lines of sight. This made sense as the diagonals are only checked if the 'next' and 'previous' values differ a large enough amount in both x and y. With a shallow line, these values may only have a large enough deviation in one plane (x or y). This was also inferred from analyzing this Bresenham simulator, which can be seen that on shallow lines the corners of cells are often missed (scroll to the bottom of the page for the simulator) https://www.middle-engine.com/blog/posts/2020/07/28/bresenhams-line-algorithm. To fix this, I just made the poison check a larger area. Instead of just checking one square with the poison-cell check, I make the program check one square on either side of the cell, as seen in images 3 and 4 below:

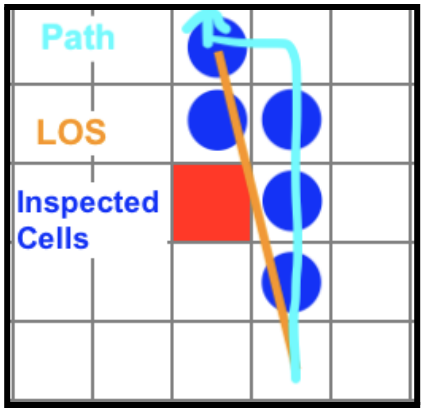

IMAGE 3: Before enlarged search

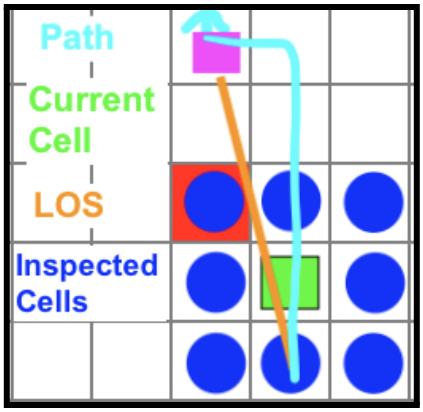

IMAGE 4: with enlarged search

- As can be seen in the image above, and in the simulator previously mentioned, the corner cutting behaviour is likely due to a shallow line of sight, making the poison not detected by the line algorithm or the added diagonal check. Enlarging this search finds these cells more easily and can be seen in the program.
- The downside is when the ant is near poison, its path following looks rigid and grid-like, while elsewhere it follows a 'normal' trajectory. The upside is this seems to have eradicated the corner cutting behaviour.

PATH FOLLOWING LOGIC:

- The ant follows the path home by simply updating its velocities in the x and y direction to travel towards the next target cell of the path. Once it arrives home, it indicates this to the calling function to trigger a state switch to wandering and the creation of a new ant.
- This allows the ant to follow closely when navigating the path around poison, but also choose a smooth trajectory when travelling smoothly between two points in the open.
- I have added a red outline to the cells in the ant's path to show where the ant is headed. These disappear in order when the ant visits them on their journey.

GAME END

- The game runs continuously, with the ant population growing as food is brought to the colony, and shrinking as ants hit the poison.
- If all ants hit the poison and die, the game stops and indicates this to the user.

PART 2: Game Compilation Instructions

SOUND AND IMAGE FILES:

- There are no sound and image files used within my game, I chose to rely on primitive shapes provided in JavaFX.

IF YOU ARE USING JAVA 8:

- Since JavaFX is included with Java8, you can simply navigate to the directory where you have saved the jar file and run:
    - java -jar A2P2_452_Dixon_3397649.jar

IF YOU ARE USING JAVA 11 and HIGHER:

- This project was developed and tested on a MacOS device using Java 17, meaning I needed to download and install JavaFX. I used JavaFX version 21.0.9.
- To run the .jar file, save the file to your device in a known location.
- Ensure JavaFX is downloaded, and find the path to the JavaFX 'lib' folder.
- In your command line interface (CLI), Navigate to the location where you saved the .jar file.
    - Now we tell java what libraries to include from JavaFX, as well as where to find them (i.e. the path to "lib" mentioned above), and then run the program.
    - An example of this is:

```
desktop % java \
--module-path /Users/bd/Desktop/AU/COMP_452/javafx/lib \
--add-modules javafx.controls,javafx.graphics \
-jar A2P2_452_Dixon_3397649.jar
```

        - Where the file (A2P2_452_Dixon_3397649.jar) is stored on on my desktop, the path to my javaFX library is /Users/bd/Desktop/AU/COMP_452/javafx/lib, and we are using the modules javafx.controls,javafx.graphics from the JavaFX library to run the program.

PART 3: Game Play Instructions

- When the program begins, it will ask for an integer between 1 and 500.
- Simply enter an integer, click submit and the game will begin.

PART 4: State Machine Description

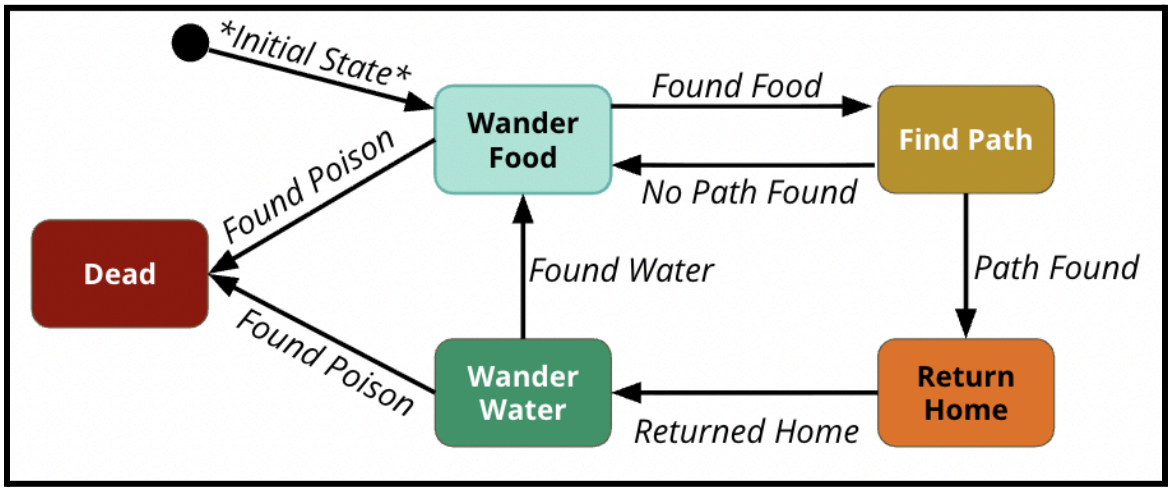

IMAGE 5: Ant State Machine Diagram

To describe my state machine, I will go through each of my state machine Java Classes (which are all currently in AntStateMachine.java) from my source code and describe what each does and how they relate to one another. All classes were kept in the same .java file for simplicity, but could easily be separated into separate files keeping the structure as is.

AntStateMachine Class

- This is the driver of my state machine, which holds an instance of each of my ant states and tracks the current state.
- When the ant is switching states, it calls this class's "switchCurrentState" and indicates which state it would like to switch to. The switched state is then set as the current state.
- The current state is then used (via polymorphism) when this class's enterCurrentState() and runCurrentState() functions are run.

AntState Abstract Class

- This class acts as a blueprint for all of my ant states.
- With this design my state machine remains modular, making it easy to separate states as well as add or remove states.
- Each state will inherit 3 methods:
    - enterState()
        - This is what needs to be done before the state is run.
        - It is called before the state's update() is run.
        - In my code, it is typically used to reset any variables used in the update() method.
    - update(double deltaTime)
        - This is what is called to run the state.
        - When a transition is met, the exitState() is run.
    - exitState()
        - This is what is run when a transition is met.

        - In my code, this is where I handled transitions and called the "switchCurrentState" and "enterCurrentState" functions to transition to the next state.
        - I initially had a Transition class to handle this, but it seemed unnecessary for this game.

<u>WanderFood, WanderWater, FindPath, ReturnHome, Dead Classes</u>

- All of these classes extend the AntState Class, and follow the enter, update, exit design described above. Their state transitions are outlined in the State Diagram above (Image 5).
- WanderFood
    - This state wanders until it finds food, when it does, it triggers FindPath.
    - If at any point the ant finds poison, it dies.
- FindPath
    - This state plays the "grabbing food animation", while also running iterations of the A* algorithm. When a path is found, it triggers the ReturnHome State.
    - The path returned has been smoothed using Bresenham's Line algorithm, with additional object inflation to remove corner cutting.
        - If somehow a path was not found (theoretically impossible for this game, but if this was a dynamic environment), it would return to WanderFood. This way if there was no path home, the ants would wander until they eventually hit poison.
- ReturnHome
    - This gets the ant character to follow the path found back to the home colony.
    - This path avoids poison.
    - Once home it triggers the WanderWater State.
- WanderWater
    - In this state, the ant wanders until it finds water. When it does, the "water drinking" animation plays, then the ant returns to WanderFood.
    - If at any point the ant finds poison, it dies.
- Dead
    - If at any point a wandering ant collides with poison, it dies and enters this state and is removed from the update loop.

PART 5: Current Game Bugs/ Problems

- PROBLEMS/LIMITATIONS:
    - When the ant is following the smoothed path, when it is near a poison cell it has quite a rigid path that it follows. This is not as clean as I would like it, but it avoids corner cutting. Implementing ray casting or another path smoothing logic could be added to improve on this.
    - During path following, I make it so that the next cell the ant is going to is highlighted in a red border. However, since all ants use the same instance of the grid, multiple ants may override this so it can turn on and off by any ant following that same path from the same food cell. This does not affect game play, but changes the visual timing.
    - The shapes used are circles and not the shape of an ant, but if looking at it as if you are viewing ants from a distance, it does appear like ants.
    - When ants die, I chose to have them displayed as dots sitting beside the poison. This can make the ant look like it is walking through the poison if it walks beside the poison cell. The ant does not actually touch the poison, just the already killed ants, but visually this may not appear correct at times.
    - Adding animations and sound may enhance the animation.
    - The colours and animations used are for visual purposes, but do not represent the actual colours seen in the regular environment.
    - There is not reset button so the program must be rerun for the user to implement another implementation.

- BUGS:
    - I have not run into any known crashes or performance issues when testing this game code.